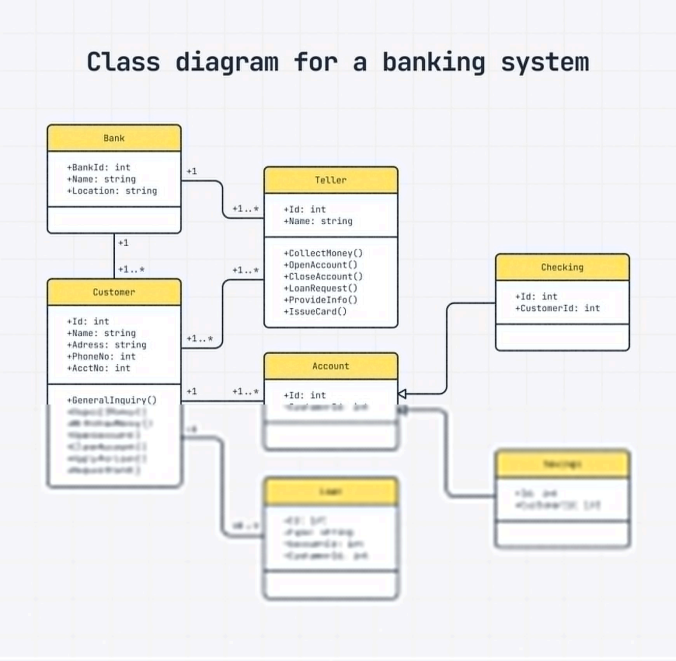
Desvendando o NestJS

POO, MVC e o Poder dos Decorators



Parte 1: A Base de Tudo

Programação Orientada a Objetos (POO)

O NestJS é construído sobre o TypeScript e incentiva fortemente o uso dos princípios da POO.

Tudo é uma Classe! (Controllers, Services, Modules...)

Os 4 Pilares da POO no NestJS

- **Encapsulamento:** Agrupar dados e métodos.
  - **Exemplo:** Um `Service` que contém toda a lógica de negócio para um recurso. O `Controller` não sabe *como* funciona, apenas *o que* ele faz.
- **Abstração:** Focar no essencial, esconder o complexo.
  - **Exemplo:** A assinatura do método `create(data)` é a abstração. A implementação é o detalhe.

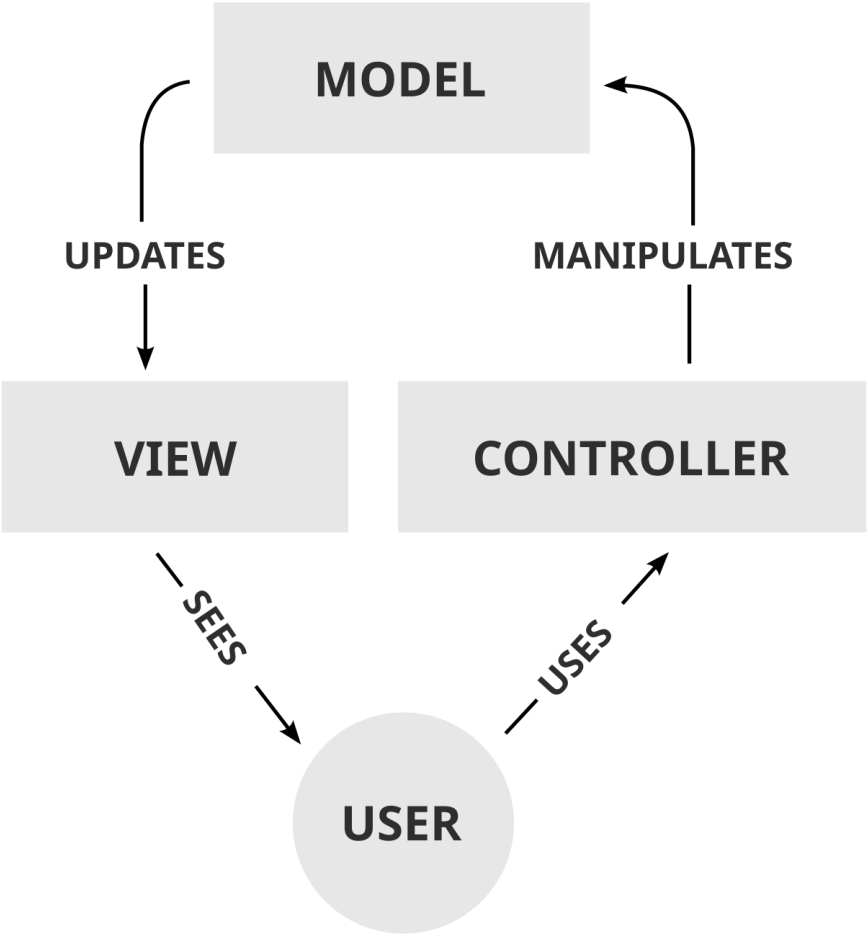
- **Herança:** Reutilizar código criando classes "filhas".
  - **Exemplo:** Um `BaseController` com métodos CRUD genéricos.
- **Polimorfismo:** "Muitas formas". Objetos se comportando de maneiras diferentes.
  - **Exemplo:** Diferentes `Loggers` (`FileLogger`, `DatabaseLogger`) que implementam a mesma interface.

Parte 2: O Padrão Arquitetural

MVC (Model-View-Controller)

Um padrão para separar as responsabilidades da aplicação.

- **Model:** Os dados e a lógica de negócio.
- **View:** A camada de apresentação (a UI).
- **Controller:** O intermediário que recebe a entrada do usuário e orquestra a comunicação.



O MVC no Contexto do NestJS (APIs)

Em uma API, a **"View"** é a resposta que enviamos, geralmente em **JSON**.

- **Controller (`@Controller`)**
  - Recebe requisições HTTP e envia respostas. É o ponto de entrada.
- **Provider/Service (`@Injectable`)**
  - Contém a lógica de negócio. É o nosso **Model** funcional.
- **Model (Entidades/DTOs)**
  - A estrutura dos dados, definida por `classes` ou `interfaces`.

Fluxo de uma Requisição no NestJS

1. **Requisição HTTP** (`GET /users/1`)
2. **Roteamento** do NestJS encontra o `Controller` correto.
3. O `Controller` chama um método no `Service` injetado.
4. O `Service` executa a lógica (ex: busca no banco de dados).
5. O `Service` retorna os dados para o `Controller`.
6. O `Controller` envia a **Resposta JSON** de volta para o cliente.

Parte 3: A Mágica

Decorators (Anotações)

Pense neles como "etiquetas" que dão instruções especiais ao NestJS.

São funções que começam com `@` e são anexadas a classes, métodos ou propriedades para adicionar metadados ou modificar seu comportamento.

```
@Controller('tasks') // <-- Isto é um Decorator!
export class TasksController {
  // ...
}
```

Decorators Essenciais

De Classe

- `@Controller('prefixo')`: Marca uma classe como um Controller.
- `@Injectable()`: Marca uma classe como um Provider (Service) que pode ser injetado em outras.

De Método (HTTP)

- `@Get()`, `@Post()`, `@Put()`, `@Patch()`, `@Delete()`: Mapeiam um método a um verbo HTTP.

De Parâmetro de Método

- `@Body()`: Extrai o corpo (payload) da requisição.
- `@Param('id')`: Extrai um parâmetro da URL (`/tasks/:id`).
- `@Query('status')`: Extrai um parâmetro da query string (`/tasks?status=open`).

Parte 4: Juntando Tudo

Exemplo Prático

1. Model (DTO - Data Transfer Object)

`create-task.dto.ts`

```
// Uma classe simples que define a estrutura dos dados
export class CreateTaskDto {
  title: string;
  description: string;
}
```

Exemplo Prático

2. Service (Lógica de Negócio)

`tasks.service.ts`

```
import { Injectable } from '@nestjs/common';
import { CreateTaskDto } from './dto/create-task.dto';

@Injectable() // Marcando como injetável
export class TasksService {
  private readonly tasks = [];

  // Método que encapsula a lógica
  create(task: CreateTaskDto) {
    this.tasks.push(task);
    return task;
  }
}
```

Exemplo Prático

3. Controller (Ponto de Entrada)

`tasks.controller.ts`

```
import { Controller, Post, Body } from '@nestjs/common';
import { CreateTaskDto } from './dto/create-task.dto';
import { TasksService } from './tasks.service';

@Controller('tasks') // Rota base: /tasks
export class TasksController {
  // Injeção de Dependência do nosso Service
  constructor(private readonly tasksService: TasksService) {}

  @Post() // Responde a: POST /tasks
  create(@Body() createTaskDto: CreateTaskDto) {
    // Delega a lógica para o Service
    return this.tasksService.create(createTaskDto);
  }
}
```

Conclusão

- **POO:** A base que nos dá `Classes` para organizar tudo.
- **MVC:** O padrão que nos ensina a separar `Controllers` (rotas), `Services` (lógica) e `Models` (dados).
- **Decorators:** A "cola" declarativa que diz ao NestJS como tudo funciona junto.

Dominar esses três conceitos é o caminho para construir aplicações robustas e escaláveis com NestJS.

Perguntas?